

Comprehensive Study Notes

Chapters 1 – 12 | 5% Weightage

1. Introduction to OS	7. Deadlocks
2. OS Structures	8. Memory Management
3. Process Management	9. File System Management
4. CPU Scheduling	10. Secondary Storage
5. Thread Management	11. I/O Systems
6. Concurrency & Sync	12. Protection & Security

Introduction to Operating Systems

1.1 What is an Operating System?

An **Operating System (OS)** is system software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between users and the computer hardware.

Key Point: The OS is the most important system software; without it, no application software can run.

1.2 Goals of an Operating System

- **Convenience:** Makes the computer easy to use for the end user.
- **Efficiency:** Manages resources (CPU, memory, I/O) efficiently.
- **Ability to Evolve:** Allows introduction of new system functions without interfering with existing ones.

1.3 Types of Operating Systems

Type	Description	Examples
Batch OS	Jobs processed in batches without user interaction.	IBM OS/360
Time-Sharing	Multiple users share CPU time simultaneously.	UNIX, Linux
Distributed	Multiple machines work together; appear as one system.	LOCUS, Amoeba
Real-Time	Strict time constraints; used in critical systems.	VxWorks, RTOS
Mobile OS	Designed for smartphones and tablets.	Android, iOS
Embedded OS	Runs on embedded devices with limited resources.	FreeRTOS, QNX

1.4 Functions of an Operating System

- **Process Management:** Creation, scheduling, termination of processes.
- **Memory Management:** Allocation and deallocation of memory space.
- **File System Management:** Organising, storing, and retrieving files.
- **I/O Device Management:** Managing input/output devices via drivers.
- **Security & Protection:** Protecting data and resources from unauthorised access.
- **Network Management:** Facilitating communication between processes/systems.

1.5 OS as a Resource Manager

The OS must allocate resources to competing processes efficiently and fairly. Resources include CPU time, memory, I/O devices, and files. The OS uses scheduling algorithms and policies to resolve conflicts.

CHAPTER 2

Operating System Structures

2.1 OS Services

An OS provides an environment for execution of programs. Key services include:

- User Interface (CLI, GUI, Batch)
- Program execution – load into memory and run
- I/O Operations – files and I/O devices
- File-system manipulation – read, write, create, delete
- Communications – shared memory or message passing
- Error detection – CPU, memory, I/O, and user-program errors
- Resource allocation – multiple users/jobs sharing resources
- Accounting – tracking resource usage
- Protection and security

2.2 System Calls

System calls provide the interface between a running program and the OS. They are typically available as routines written in C/C++. Categories:

Category	Examples
Process Control	fork(), exit(), wait(), exec()
File Management	open(), read(), write(), close(), unlink()
Device Management	ioctl(), read(), write()
Information	getpid(), alarm(), sleep()
Communication	pipe(), shmget(), mmap(), socket()
Protection	chmod(), chown(), umask()

2.3 OS Design Approaches / Structures

Simple / Monolithic Structure

All OS functionality in one large kernel. Fast but difficult to maintain. *Example: early UNIX.*

Layered Approach

OS divided into layers; each layer uses services only of lower layers. Easier to debug but less efficient.

Microkernel

Minimal kernel; most services run in user space as servers. More reliable & portable. *Example: Mach, QNX.*

Modular Kernel

Core kernel with loadable modules. Modern Linux uses this. Flexible and efficient.

Exokernel

Kernel exposes hardware directly; OS abstractions in user-level libraries. Maximum performance.

Virtual Machine (VM)

Hardware virtualised; multiple OS instances share one physical machine. *Example: VMware, KVM.*

Process Management

3.1 What is a Process?

A **process** is a program in execution. It is an active entity, whereas a program is a passive entity stored on disk. A process includes the program code (*text section*), current activity (program counter, registers), stack (temporary data), data section (global variables), and heap (dynamic memory).

3.2 Process States

- **New:** Process is being created.
- **Ready:** Process is waiting to be assigned to the CPU.
- **Running:** Instructions are being executed.
- **Waiting:** Process is waiting for some event (I/O completion, signal).
- **Terminated:** Process has finished execution.

Key Point: Only ONE process can be in the Running state on a single-core CPU at any time.

3.3 Process Control Block (PCB)

Each process is represented in the OS by a PCB containing:

- Process State (new, running, waiting, etc.)
- Process ID (PID)
- Program Counter (address of next instruction)
- CPU Registers (accumulators, index registers, stack pointers)
- CPU Scheduling Information (priority, pointers to scheduling queues)
- Memory Management Information (page tables, segment tables)
- Accounting Information (CPU time used, time limits, job numbers)
- I/O Status Information (list of I/O devices allocated, open files)

3.4 Process Scheduling

The OS maintains several **scheduling queues**:

- **Job Queue:** All processes in the system.
- **Ready Queue:** Processes in main memory, ready to execute.
- **Device Queues:** Processes waiting for a particular I/O device.

3.5 Context Switch

When the CPU switches from one process to another, the OS saves the state of the old process and loads the saved state of the new process. This is called a **context switch**. It is pure overhead – no useful work is done during switching.

3.6 Process Creation & Termination

Processes are created using **fork()** (UNIX). The child is a copy of the parent. The parent can wait for the child using **wait()**. A process terminates via **exit()**. If a parent exits before child, child becomes an **orphan** (adopted by init/PID 1). A terminated but not yet reaped process is a **zombie**.

3.7 Interprocess Communication (IPC)

Two main models:

- **Shared Memory:** Processes share a region of memory. Fast but requires synchronisation.
- **Message Passing:** Processes communicate via `send()/receive()` primitives. Easier to implement in distributed systems.

CHAPTER 4

CPU Scheduling

4.1 Basic Concepts

CPU scheduling decides which process in the ready queue gets the CPU next. The goal is to maximise CPU utilisation. Scheduling occurs when a process:

- Switches from running to waiting state (I/O request)
- Switches from running to ready state (interrupt)
- Switches from waiting to ready state (I/O completion)
- Terminates

4.2 Scheduling Criteria

CPU Utilisation	Keep CPU as busy as possible (40–90%).
Throughput	Number of processes completed per time unit.
Turnaround Time	Total time from submission to completion.
Waiting Time	Time spent in the ready queue.
Response Time	Time from submission of request to first response.

4.3 Scheduling Algorithms

First-Come First-Served (FCFS)

Simplest algorithm. Non-preemptive. Jobs executed in arrival order. Suffers from **convoy effect** – short jobs wait behind long ones.

Shortest Job First (SJF)

Selects process with smallest next CPU burst. Can be preemptive (SRTF). Optimal for minimising average waiting time but requires knowing burst time.

Round Robin (RR)

Each process gets a fixed time quantum (typically 10–100 ms). Preemptive. Fair. Suitable for time-sharing. Performance depends on quantum size.

Priority Scheduling

Each process assigned a priority; CPU given to highest priority process. Can cause **starvation** – solved by **aging** (gradually increase priority).

Multilevel Queue

Ready queue partitioned into separate queues (foreground/background). Each queue has its own algorithm. Fixed priority between queues.

Multilevel Feedback Queue

Most general and complex. Processes can move between queues. Allows aging, penalises CPU-bound processes. Used in most modern OSes.

4.4 Multi-Processor Scheduling

With multiple CPUs, load sharing is possible. Issues include **processor affinity** (keep process on same CPU to exploit cache), **load balancing** (push/pull migration), and **NUMA** (Non-Uniform Memory Access) awareness.

Thread Management

5.1 What is a Thread?

A **thread** (lightweight process) is a basic unit of CPU utilisation. It comprises a thread ID, program counter, register set, and stack. Threads within the same process share code, data, and OS resources (files, signals).

Key Point: Threads share the heap and global variables but have their own stack and registers.

5.2 Benefits of Multithreading

- **Responsiveness:** UI thread stays responsive while worker threads process.
- **Resource Sharing:** Threads share memory easily (unlike processes needing IPC).
- **Economy:** Thread creation is 30× cheaper than process creation.
- **Scalability:** Threads can run truly in parallel on multicore CPUs.

5.3 Multithreading Models

- **Many-to-One:** Many user threads → one kernel thread. Blocking call blocks all. No true parallelism.
- **One-to-One:** Each user thread → one kernel thread. True parallelism. Overhead per thread. *Linux, Windows.*
- **Many-to-Many:** Many user threads multiplexed to many kernel threads. Best of both worlds. Complex to implement.
- **Two-Level:** Variation of M:N – allows a user thread to be bound to one kernel thread.

5.4 Thread Libraries

- **POSIX Pthreads:** Standard UNIX/Linux threading library.
- **Java Threads:** Platform-independent via JVM.
- **Win32 Threads:** Windows native threading API.

5.5 Thread Issues

- **fork() & exec():** Does fork() duplicate all threads or just the calling one? UNIX: two versions.
- **Thread Cancellation:** Terminate before completion. Asynchronous (immediate) or deferred (check cancellation point).
- **Signal Handling:** Where is the signal delivered in a multithreaded process? Synchronous → faulting thread.
- **Thread Pools:** Pre-created threads waiting for work. Faster than creating on demand; bounds thread count.
- **Thread-Local Storage:** Each thread has its own copy of data (TLS). Similar to static data.

Concurrency & Synchronisation

6.1 The Critical Section Problem

A **critical section** is a code segment accessing shared resources. Only one process should execute in its critical section at a time. Requirements for a solution:

- **Mutual Exclusion:** Only one process in critical section at a time.
- **Progress:** If no process is in critical section, selection of next entrant cannot be postponed indefinitely.
- **Bounded Waiting:** A bound must exist on the number of times other processes enter before a waiting process.

6.2 Peterson's Solution

Software solution for **two** processes using two shared variables: **int turn** and **boolean flag[2]**. Satisfies all three requirements but requires *busy waiting*.

6.3 Synchronisation Hardware

- **Memory Barriers:** Ensure memory operations complete in order.
- **Hardware Instructions:** Atomic **test_and_set()** and **compare_and_swap()**.
- **Atomic Variables:** Operations on integers/booleans done atomically (e.g., C++ **std::atomic**).

6.4 Mutex Locks

A **mutex** (mutual exclusion lock) provides **acquire()** and **release()** operations. Simple but *spinlocks* waste CPU (busy waiting). Good for multiprocessor systems where lock hold time is short.

6.5 Semaphores

A **semaphore** **S** is an integer variable with two atomic operations: **wait(S)** (decrement; block if $S \leq 0$) and **signal(S)** (increment; wake blocked process).

- **Binary Semaphore (Mutex):** Value 0 or 1. Same as mutex lock.
- **Counting Semaphore:** Value ranges over unrestricted domain. Controls access to finite resource pool.

Key Point: Semaphores can cause deadlock and starvation. Monitor or condition variables are preferred in high-level design.

6.6 Classic Synchronisation Problems

Bounded-Buffer (Producer-Consumer)

Producer adds items; consumer removes. Use 3 semaphores: mutex, empty, full.

Readers-Writers Problem

Multiple readers OK simultaneously; writers need exclusive access. Risk of writer starvation.

Dining Philosophers Problem

5 philosophers, 5 chopsticks. Must avoid deadlock and starvation while sharing resources.

6.7 Monitors

A **monitor** is a high-level synchronisation construct. Only one process can be active in a monitor at a time. Uses **condition variables** with **wait()** and **signal()**. Easier to reason about than semaphores. Used in Java (synchronized blocks).

CHAPTER 7

Deadlocks

7.1 Definition

A **deadlock** is a situation where a set of processes are each waiting for an event that can be caused only by another process in the set – resulting in permanent blocking.

7.2 Necessary Conditions (Coffman's Conditions)

ALL four must hold simultaneously for deadlock to occur:

- **C1. Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
- **C2. Hold and Wait:** A process holding resources can request more resources.
- **C3. No Preemption:** Resources cannot be forcibly taken; must be released voluntarily.
- **C4. Circular Wait:** A circular chain of processes, each waiting for a resource held by the next.

7.3 Resource Allocation Graph (RAG)

Circles = processes; rectangles = resource types; dots inside = instances. Request edge: $P \rightarrow R$. Assignment edge: $R \rightarrow P$. If graph has **no cycle** $\rightarrow$ no deadlock. If graph has **a cycle** $\rightarrow$ deadlock *if* each resource type has exactly one instance.

7.4 Handling Deadlocks

Prevention

Ensure at least one Coffman condition never holds. E.g., impose total ordering on resources (eliminates circular wait).

Avoidance

Use Banker's Algorithm: grant resource only if system remains in a **safe state**. Requires advance knowledge of maximum resource needs.

Detection & Recovery

Allow deadlock, detect via RAG or detection algorithm, then recover by process termination or resource preemption.

Ignorance (Ostrich Algorithm)

Pretend deadlocks don't occur. Acceptable when deadlocks are very rare. Used in most OSes (Windows, Linux) for user-level processes.

7.5 Banker's Algorithm

Used for **deadlock avoidance** with multiple resource types. Checks if granting a request keeps the system in a safe state by simulating resource allocation. A **safe sequence** must exist where every process can eventually finish.

Key Point: Banker's Algorithm is theoretically important but rarely used in practice due to its overhead and requirement of knowing resource needs in advance.

CHAPTER 8

Memory Management

8.1 Background

Memory management ensures processes are efficiently allocated memory. Key concepts: **logical address** (generated by CPU / virtual address) vs **physical address** (actual RAM location). The **Memory Management Unit (MMU)** maps virtual to physical addresses at runtime.

8.2 Contiguous Memory Allocation

- **Fixed Partitioning**: Memory divided into fixed-size partitions. Causes internal fragmentation.
- **Variable Partitioning**: Partitions sized to process needs. Causes external fragmentation. Strategies: First-Fit, Best-Fit, Worst-Fit.

Compaction can reduce external fragmentation but is expensive. **Internal fragmentation** = wasted space within a partition. **External fragmentation** = free memory but not contiguous.

8.3 Paging

Physical memory divided into fixed-size **frames**; logical memory into same-size **pages**. OS maintains a **page table** mapping pages to frames. Eliminates external fragmentation; may have internal fragmentation (last page).

- **TLB (Translation Lookaside Buffer)**: Cache for page table entries. Provides fast address translation. Hit ratio ~98%.
- **Multi-level Paging**: For large address spaces, page the page table itself (2-level, 3-level).
- **Inverted Page Table**: One entry per physical frame. Saves space but searching is slow.

8.4 Segmentation

Memory viewed as variable-length **segments** (code, data, stack). Logical address = (segment number, offset). OS maintains **segment table**. Provides user's view of memory. May cause external fragmentation.

8.5 Virtual Memory

Allows execution of programs that are not completely in memory. The full program need not be loaded; only active pages loaded on demand (**demand paging**). Benefits: run larger programs, higher multiprogramming degree.

8.6 Page Replacement Algorithms

- **FIFO**: Replace oldest page in memory. Simple but suffers Belady's Anomaly.
- **Optimal (OPT)**: Replace page not used for longest future time. Best performance but not implementable (requires future knowledge).
- **LRU**: Replace page not used for longest past time. Good performance. Implemented via counters or stack.
- **LRU Approximation**: Second-chance / Clock algorithm. Uses reference bit to approximate LRU. Practical.
- **MFU / LFU**: Most/Least Frequently Used. Less common. Idea: heavily used pages are likely needed again.

8.7 Thrashing

Thrashing: A process spends more time paging than executing. Occurs when a process doesn't have enough frames. Detected when CPU utilisation drops while paging increases. Solution: **Working Set Model** – allocate frames based on working set (set of pages used in last Δ references).

CHAPTER 9

File System Management

9.1 File Concepts

A **file** is a named collection of related information stored on secondary storage. File attributes: name, identifier, type, location, size, protection, timestamps. File operations: create, write, read, reposition, delete, truncate.

9.2 File Types

- **Regular Files:** Data files (text, binary, executable).
- **Directories:** Files containing lists of other files.
- **Special Files:** Character devices, block devices, pipes, sockets.

9.3 Directory Structure

- **Single-Level:** All files in one directory. Simple but naming conflicts.
- **Two-Level:** Separate directory for each user. Isolates users.
- **Tree-Structured:** General tree hierarchy. Current concept. Supports absolute/relative paths.
- **Acyclic-Graph:** Shared subdirectories/files via links. Aliases allowed.
- **General Graph:** Cycles possible; need garbage collection. Rare.

9.4 File Allocation Methods

Contiguous Allocation

Each file occupies contiguous blocks. Fast sequential/random access. External fragmentation; file size must be known.

Linked Allocation

Each block contains pointer to next. No external fragmentation. Poor random access; pointer overhead. Variant: FAT (File Allocation Table).

Indexed Allocation

Index block contains pointers to file blocks. Supports random access. Index block overhead. Multi-level indexing for large files. Used in Unix (inodes).

9.5 Free Space Management

- **Bit Vector (Bitmap):** One bit per block. Easy to find free blocks. Space overhead.
- **Linked List:** Chain all free blocks. Traverse needed to find contiguous space.
- **Grouping:** First free block stores addresses of n free blocks.
- **Counting:** Store address of first free block + count of following free blocks.

9.6 Common File Systems

File System	OS	Features
FAT32	Windows/Legacy	Simple; max 4 GB file size

NTFS	Windows	Journaling, ACLs, compression, encryption
ext4	Linux	Journaling, extents, large files
APFS	macOS/iOS	SSD-optimised, snapshots, encryption
ZFS	Solaris/Linux	128-bit, pooled storage, copy-on-write

Secondary Storage Management

10.1 Disk Structure

A **hard disk drive (HDD)** consists of platters, tracks, sectors, and cylinders. Data access time = **seek time** (move arm) + **rotational latency** (wait for sector) + **transfer time**.

10.2 Disk Scheduling Algorithms

- **FCFS**: Service requests in arrival order. Simple but high seek time.
- **SSTF**: Shortest Seek Time First. Go to nearest request. May cause starvation.
- **SCAN**: Arm sweeps disk like elevator; services requests in each direction. Fair.
- **C-SCAN**: Circular SCAN. Sweep one direction only; return to start. More uniform wait.
- **LOOK**: Like SCAN but only goes as far as last request (doesn't go to edge).
- **C-LOOK**: Circular LOOK. Most commonly used in modern OSes.

10.3 Disk Management

- **Low-level Formatting**: Divides disk into sectors; done at factory.
- **Partitioning**: Divides disk into groups of cylinders (logical disks).
- **Logical Formatting**: Creates file system on partition.
- **Boot Block**: Bootstrap loader in boot sector (MBR or UEFI).
- **Bad Block Management**: Sector sparing – map bad sectors to spare sectors.

10.4 RAID (Redundant Array of Independent Disks)

Level	Description	Advantage
RAID 0	Striping – data spread across disks	High performance; no redundancy
RAID 1	Mirroring – duplicate data	Full redundancy; expensive
RAID 4	Block striping with dedicated parity disk	Good reads; parity bottleneck
RAID 5	Block striping with distributed parity	Good reads; most popular
RAID 6	Double distributed parity	Survives 2 disk failures
RAID 10	Stripe of mirrors (RAID 1+0)	High performance + redundancy

10.5 Solid State Drives (SSDs)

SSDs use flash memory. No moving parts → faster, quieter, more durable. No seek/rotational latency. But: **write amplification**, limited write cycles. Use **wear levelling** to distribute writes evenly. Disk scheduling less critical – FCFS often used for SSDs.

Input / Output Systems

11.1 I/O Hardware

I/O devices communicate via **ports**, **buses**, and **device controllers**. The CPU issues commands to controllers via **control registers** and **data buffers**. Modern systems use memory-mapped I/O.

- **Port:** Connection point (e.g., USB, serial port).
- **Bus:** Shared communication path (PCI, USB, SATA).
- **Controller:** Electronic component that operates port/bus/device (has registers, buffer).

11.2 I/O Communication Techniques

Polling (Busy-Wait)

CPU continuously checks device status register. Simple but wastes CPU cycles.

Interrupt-Driven I/O

Device interrupts CPU when ready. CPU free to do other work. Most common.

Direct Memory Access (DMA)

Dedicated DMA controller transfers data between device and memory without CPU intervention. CPU only involved at start and end. Most efficient for bulk transfers.

11.3 I/O Software Layers

- **Hardware:** Physical devices and their controllers.
- **Interrupt Handlers:** Handle I/O completions, wake up sleeping driver.
- **Device Drivers:** Interface between OS and specific hardware. Translate generic I/O requests to device commands.
- **Device-Independent OS Software:** Buffering, caching, error reporting, scheduling, block size management.
- **User-level I/O Software:** Library routines (printf, scanf). Standardises I/O interface.

11.4 Buffering, Caching, and Spooling

- **Buffering:** Temporary storage of data during transfer. Handles speed mismatch and different transfer sizes.
- **Caching:** Copy of data in faster storage. Improves performance by reusing recently accessed data.
- **Spooling:** Simultaneous Peripheral Operations Online. Queue jobs for devices that can't accept interleaved output (printers).

11.5 I/O Scheduling

I/O requests can be queued and scheduled to improve efficiency. For disks, scheduling algorithms (SSTF, SCAN) are applied. Goal: minimise latency and maximise throughput.

11.6 Performance Considerations

I/O is often the performance bottleneck. Strategies: reduce context switches, use DMA, use large transfers, use smarter hardware (intelligent controllers), balance CPU and I/O activity.

Protection & Security

12.1 Goals of Protection

Protection refers to mechanisms for controlling access of processes/users to resources. Goals:

- Prevent malicious misuse of resources.
- Detect and neutralise latent errors.
- Ensure each process uses resources only in authorised ways.
- Support sharing while ensuring privacy.

12.2 Principles of Protection

- **Principle of Least Privilege:** Every process/user should operate with minimum necessary privileges.
- **Separation of Privilege:** Two keys required to unlock a secure state.
- **Need-to-Know:** Access only to information required for the task.
- **Compartmentalisation:** Isolate system components to limit damage.

12.3 Access Control

Access Control Matrix

2D matrix: rows = domains (users/processes), columns = objects (files, devices). Each cell lists permitted operations. Too large for practical use.

Access Control Lists (ACLs)

Each object has a list of (domain, rights) pairs. Used in most modern OSes (Linux, Windows).

Capability Lists

Each domain has a list of (object, rights) pairs. Rows of the access matrix. Used in some microkernels.

Role-Based Access Control (RBAC)

Permissions assigned to roles; users assigned to roles. Simplifies administration. Common in enterprise systems.

12.4 Security Threats

Malware	Viruses, worms, trojans, ransomware, spyware, rootkits.
Code Injection	SQL injection, buffer overflow, format string attacks.
Privilege Escalation	Gain higher privileges than intended.
Man-in-the-Middle	Intercept communication between two parties.
Denial of Service	Overwhelm system to make it unavailable.
Phishing / Social Engineering	Trick users into revealing credentials.

12.5 Cryptography in OS Security

- **Symmetric Encryption:** Same key for encrypt/decrypt (AES). Fast. Key distribution problem.
- **Asymmetric Encryption:** Public/private key pair (RSA). Slow. Solves key distribution.

- **Hash Functions:** One-way (SHA-256). Used for password storage, integrity checks.
- **Digital Signatures:** Authentication and non-repudiation.
- **TLS/SSL:** Secures network communication.

12.6 User Authentication

- **Something you know:** Passwords, PINs. Most common but weakest.
- **Something you have:** Smart cards, OTP tokens, mobile phone.
- **Something you are:** Biometrics (fingerprint, facial recognition, iris scan).
- **Multi-Factor Authentication (MFA):** Combines two or more factors. Strongly recommended.

12.7 OS Security Mechanisms

- **Process Isolation:** Each process has separate virtual address space.
- **Mandatory Access Control (MAC):** System-enforced policies (SELinux, AppArmor).
- **Discretionary Access Control (DAC):** Owner controls access (standard Unix permissions).
- **Auditing and Logging:** Track system calls, login attempts, file access.
- **Firewalls and IDS:** Control network access and detect intrusion.
- **Sandboxing:** Restrict application's access to system resources (containers, VMs).

Key Point: Security is a chain – the OS must protect itself (kernel), its users (authentication), their data (access control), and communication (encryption). A break at any link compromises the whole system.